

PROJECT REPORT

SERVERLESS STUDENT LOGIN PORTAL WITH DYNAMODB INTEGRATION

1. PROJECT OVERVIEW

This project demonstrates the implementation of a secure, production-ready, fully Serverless Full-Stack Web Application hosted on AWS. The application provides a portal where students can enter their login credentials. Upon validation against a live NoSQL database, users are dynamically redirected to a private dashboard containing customized content regarding friendship relationships.

2. SYSTEM ARCHITECTURE

The application is built entirely on a serverless architecture, meaning zero infrastructure management, automatic scaling, and high availability.

Components Used:

- * AWS Amplify & Amazon S3: Hosts the static web assets (index.html and dashboard.html) globally, serving them securely via HTTPS.
- * AWS API Gateway: Acts as the secure entrance endpoint for frontend requests. It manages cross-origin traffic regulations (CORS) and routes login payloads securely to the backend.
- * AWS Lambda: Runs an isolated Python 3.12 function on-demand to process requests, communicate with the database, evaluate authentication details, and structure responses.

* Amazon DynamoDB: A serverless NoSQL database storing user entries using a key-value pattern. It scales automatically to handle infinite query lookups.

3. STEP-BY-STEP IMPLEMENTATION GUIDE

Step 1: Database Creation (Amazon DynamoDB)

1. Opened the DynamoDB Console and clicked Create table.
2. Set Table name to "Students".
3. Set Partition key to "username" (Type: String).
4. Clicked Create table and waited for status to become Active.
5. Navigated to Actions > Explore items and clicked Create item.
6. Added a primary string entry for username (e.g., "naga").
7. Clicked Add new attribute > String, named it "password", and provided a value (e.g., "naga123").
8. Clicked Create item to save the record.

Step 2: Access Control Configuration (AWS IAM)

1. Navigated to the AWS Lambda Console and selected the main function.
2. Under Configuration > Permissions, clicked the active Execution Role link to open the IAM console.
3. Clicked Add permissions > Attach policies.
4. Searched for and selected "AmazonDynamoDBReadOnlyAccess".
5. Saved the changes, establishing secure backend-to-database communication based on the principle of least privilege.

Step 3: Backend Logic Deployment (AWS Lambda)

1. Opened AWS Lambda, created a function named "studentLoginFunction", and chose Python 3.12 as the runtime.
2. Erased the default code template and deployed the database search script.

Step 4: Endpoint Routing Setup (AWS API Gateway)

1. Opened API Gateway and created a new REST API named "StudentLoginAPI".
2. Created a target resource path named "/login".
3. Configured a POST method linked directly to "studentLoginFunction".
4. Clicked Enable CORS on the "/login" resource to safely authorize inbound browser requests coming from external hosting domains.
5. Deployed the API to a stage named "prod" and saved the generated Invoke URL.

Step 5: Frontend Interface Development

Created two static source files on a local desktop workspace:

- * index.html (Handles user inputs, API communication, and conditional routing)
- * dashboard.html (Houses restricted access student resources regarding Friendship Relationships)

Step 6: Application Deployment (AWS Amplify)

1. Opened the target desktop directory containing both index.html and dashboard.html sitting directly side-by-side.
 2. Selected both files simultaneously and compressed them directly into a root-level archive.zip.
 3. Opened AWS Amplify, selected Deploy without Git provider, named the application, and uploaded the zip package.
 4. Amplify unzipped the files and deployed them to a live, secure URL.
-

4. KEY TROUBLESHOOTING ACCOMPLISHED

- * CORS Management: Resolved Cross-Origin Resource Sharing restrictions in API Gateway by configuring an OPTIONS preflight method and writing matching access headers explicitly into the Lambda Python dictionaries ('Access-Control-Allow-Origin': '*').
- * Proxy Payloads Unwrapping: Rectified the initial "undefined" text display error by building a dynamic validation block inside the frontend JavaScript routine to properly parse nested proxy objects coming from AWS.

5. PROJECT CONCLUSION

The system successfully functions as intended. It demonstrates modern web engineering practices including modular serverless processing, independent client-side scripting, persistent data retrieval, and secure cloud resource connectivity.

Aws lambda code

```
import json

import boto3

# Initialize the DynamoDB connector outside the handler for fast performance
dynamodb = boto3.resource('dynamodb')

table = dynamodb.Table('Students')

def lambda_handler(event, context):
    try:
        # Extract frontend data safely depending on how API Gateway bundles it
        if 'body' in event and event['body']:
            body = json.loads(event['body'])
        else:
            body = event

        input_username = body.get('username', '').strip()
        input_password = body.get('password', '').strip()

    except Exception as e:
        return {
            'statusCode': 400,
            'headers': {
                'Access-Control-Allow-Origin': '*',
                'Access-Control-Allow-Headers': 'Content-Type',
                'Access-Control-Allow-Methods': 'OPTIONS,POST'
            },
        }
```

```
    'body': json.dumps({'message': 'Invalid input data format.'})
}
```

Verify fields are not blank

if not input_username or not input_password:

```
    return {
        'statusCode': 400,
        'headers': {
            'Access-Control-Allow-Origin': '*',
            'Access-Control-Allow-Headers': 'Content-Type'
        },
        'body': json.dumps({'message': 'Please enter both username and password.'})
    }
```

try:

Search the DynamoDB 'Students' table using the username

response = table.get_item(Key={'username': input_username})

Check if the user was found

if 'Item' in response:

student_record = response['Item']

db_password = student_record.get('password')

Match the typed password against the database password

if input_password == db_password:

```
    return {
        'statusCode': 200,
        'headers': {
```

```
        'Access-Control-Allow-Origin': '*',
        'Access-Control-Allow-Headers': 'Content-Type'
    },
    'body': json.dumps({'message': f'Login Successful! Welcome,
{input_username}.'})
}
```

If user doesn't exist OR password doesn't match

```
return {
    'statusCode': 401,
    'headers': {
        'Access-Control-Allow-Origin': '*',
        'Access-Control-Allow-Headers': 'Content-Type'
    },
    'body': json.dumps({'message': 'Invalid Credentials. Try again.'})
}
```

except Exception as e:

```
return {
    'statusCode': 500,
    'headers': {
        'Access-Control-Allow-Origin': '*',
        'Access-Control-Allow-Headers': 'Content-Type'
    },
    'body': json.dumps({'message': f'Database Connection Error: {str(e)}'})
}
```

Login page: index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Student Login Portal</title>

  <style>

    body { font-family: 'Segoe UI', sans-serif; background: #f0f2f5; display: flex; justify-
content: center; align-items: center; height: 100vh; margin: 0; }

    .login-box { background: white; padding: 40px; border-radius: 10px; box-shadow: 0 4px
12px rgba(0,0,0,0.1); width: 320px; }

    h2 { text-align: center; color: #333; margin-bottom: 24px; }

    input { width: 100%; padding: 12px; margin: 10px 0; box-sizing: border-box; border: 1px
solid #ccc; border-radius: 6px; }

    button { width: 100%; padding: 12px; background: #0056b3; color: white; border: none;
border-radius: 6px; cursor: pointer; font-size: 16px; font-weight: bold; }

    button:hover { background: #004494; }

    #message { margin-top: 20px; text-align: center; font-weight: 600; font-size: 15px; }

  </style>

</head>

<body>

<div class="login-box">

  <h2>Student Login</h2>

  <input type="text" id="username" placeholder="Username">

  <input type="password" id="password" placeholder="Password">

  <button onclick="login()">Login</button>

  <div id="message"></div>

</div>
```

```

<script>

async function login() {

    const u = document.getElementById('username').value;
    const p = document.getElementById('password').value;
    const messageDiv = document.getElementById('message');
    const apiUrl = "YOUR_API_GATEWAY_URL_HERE/login"; // Configured Route


    messageDiv.innerText = "Connecting...";
    messageDiv.style.color = "#666";


    try {
        const response = await fetch(apiUrl, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ username: u, password: p })
        });

        let data = await response.json();

        let displayedMessage = "";

        let isSuccess = response.status === 200;


        if (data && data.body) {
            try {
                const parsedBody = typeof data.body === 'string' ? JSON.parse(data.body) :
data.body;

                displayedMessage = parsedBody.message;
            } catch(e) { displayedMessage = data.body; }

            isSuccess = data.statusCode === 200;
        } else {

```

```
        displayedMessage = data.message || JSON.stringify(data);
    }

    if (isSuccess) {
        messageDiv.style.color = "green";
        messageDiv.innerText = displayedMessage + " Redirecting...";
        setTimeout(() => { window.location.href = "dashboard.html"; }, 1500);
    } else {
        messageDiv.style.color = "red";
        messageDiv.innerText = displayedMessage;
    }
} catch (error) {
    messageDiv.style.color = "red";
    messageDiv.innerText = "Error connecting to server.";
}
}

</script>
</body>
</html>
```

Privet content: dashboard.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Dashboard - Friendship Thoughts</title>

  <style>

    body { font-family: 'Segoe UI', sans-serif; background: #f0f2f5; margin: 0; padding:
20px; display: flex; justify-content: center; }

    .container { background: white; padding: 40px; border-radius: 10px; box-shadow: 0
4px 12px rgba(0,0,0,0.1); max-width: 600px; width: 100%; }

    h1 { color: #0056b3; }

    h3 { color: #333; border-bottom: 2px solid #0056b3; padding-bottom: 8px; }

    p { color: #555; line-height: 1.6; font-size: 16px; }

    .quote-box { background: #e7f3ff; border-left: 5px solid #0056b3; padding: 15px;
margin: 20px 0; font-style: italic; }

    .logout-btn { background: #dc3545; color: white; border: none; padding: 10px 15px;
border-radius: 5px; cursor: pointer; font-weight: bold; }

  </style>

</head>

<body>

<div class="container">

  <h1>Welcome to your Dashboard!</h1>

  <p>You have successfully logged through your secure AWS portal.</p>

  <h3>Understanding Friendship Relationships</h3>

  <p>Friendship is a beautiful connection built on trust, mutual respect, and shared
experiences. In a healthy relationship, friends uplift each other, listen without judgment,
and stand together during life's ups and downs.</p>


```

```
<div class="quote-box">"A true friend is someone who walks in when the rest of the  
world walks out."</div>
```

```
<button class="logout-btn" onclick="location.href='index.html'">Logout</button>
```

```
</div>
```

```
</body>
```

```
</html>
```